

Ain Shams University
Faculty of Engineering

CSE331 – Data Structures and Algorithms

Colored Water Sort Solver Project

Project Report

Student: Mahmoud Abdelaziz Mohammed
ID: 24p0485

A console-based C++ solver for the Colored Water Sort puzzle using Breadth-First Search (BFS).

1. Project Overview

The project is a console-based C++ solver for the Colored Water Sort puzzle. The program receives the initial tube arrangement and searches for a valid solution with the minimum number of moves. The project specification requires C++, supports a team size of one or two students, and states that a console interface is sufficient.

2. Team Responsibilities

Mahmoud Abdelaziz Mohammed: Implementation of the solver, data-structure design, testing, debugging, documentation, and GitHub repository preparation.

3. Data Structures and Why They Were Selected

Data Structure	Purpose
vector<>	Represents the complete puzzle. Each inner vector represents one tube and stores its layers from bottom to top.
struct State	Stores a puzzle state together with its parent state index and the source/destination tubes used to create it.
queue	Stores state indices for BFS. Because every pour is one move, BFS explores states in increasing move count.
vector of visited states	Keeps previously explored arrangements so the same state is not processed repeatedly.

4. Code Structure

Function / Component	Purpose
getTopColor()	Finds the top non-empty color of a tube.
getTopCount()	Counts the connected layers of the top color.
getEmptyPosition()	Finds the first empty position in a tube.
isSolved()	Checks whether every non-empty tube is full and contains one color.
canPour()	Checks whether a source-to-destination pour is valid.
pour()	Applies a valid pour to a copied state.
sameState()	Compares two puzzle arrangements.
isVisited()	Checks whether an arrangement was already explored.
solve()	Runs BFS and reconstructs the minimum-move solution.
printState()	Prints the final tube arrangement.

5. Algorithm

The solver uses Breadth-First Search (BFS). The initial arrangement is inserted into the queue. For each state, the program checks every source/destination pair, generates valid next states, ignores already visited states, and adds new states to the queue. Since each pour has a cost of one move, BFS explores states by increasing number of moves. Therefore, the first solved state found represents a minimum-move solution.

Each generated state stores the index of its parent state and the move that produced it. When a solved state is found, the parent links are followed backward and the moves are reversed to obtain the correct order.

6. Complexity Analysis

Let S be the number of reachable states, N the number of tubes, and C the tube capacity. Each state considers $O(N^2)$ possible source/destination pairs. State comparisons and the simple visited-state search can require $O(N \cdot C)$ work per stored state. With the vector-based visited structure, the worst-case visited checking can reach $O(S \cdot N \cdot C)$ for one generated state. Overall, the practical running time depends on the number of reachable states and can grow quickly for larger puzzles.

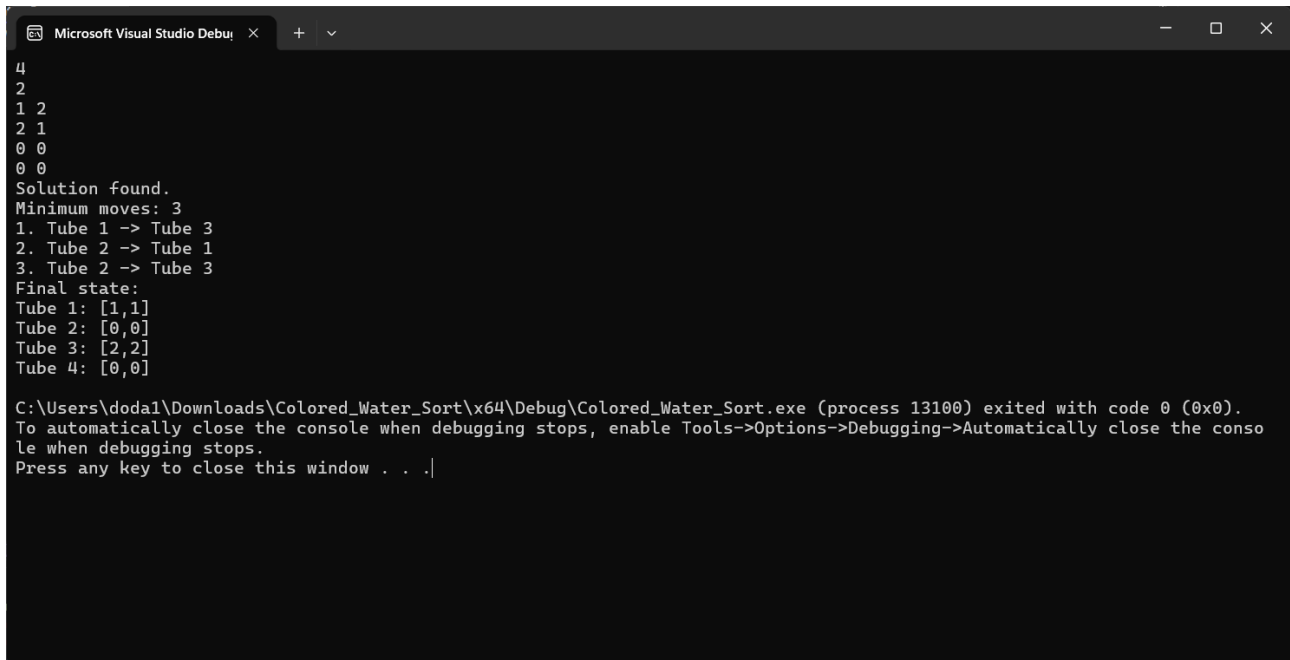
Memory usage is $O(S \cdot N \cdot C)$ for the stored states and visited arrangements, plus the BFS queue. The parent information is stored with each state so the solution path can be reconstructed without storing the full move sequence in every state.

7. Test Cases and Results

The following screenshots show the actual program output from Visual Studio for the three test cases. They cover a solvable official example, the official unsolvable example, and an additional solvable test.

Test Case 1 — Official Example A

Expected: Solution found with minimum 3 moves.



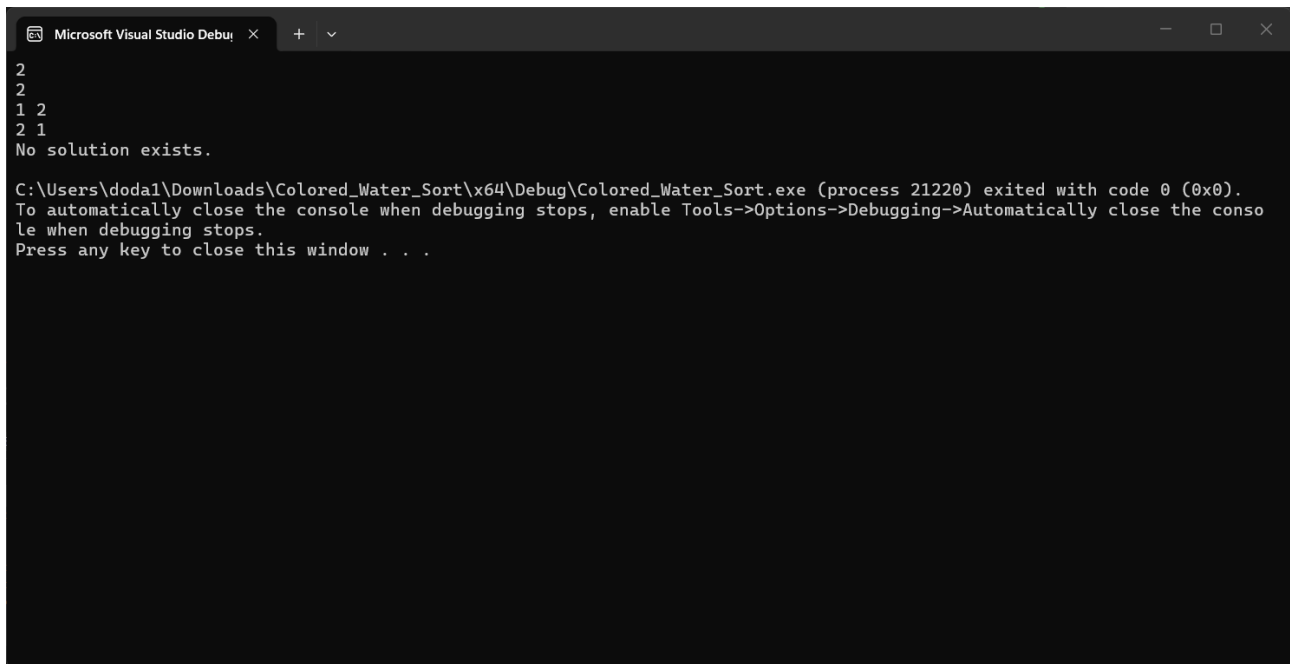
```
Microsoft Visual Studio Debug Console
4
2
1 2
2 1
0 0
0 0
Solution found.
Minimum moves: 3
1. Tube 1 -> Tube 3
2. Tube 2 -> Tube 1
3. Tube 2 -> Tube 3
Final state:
Tube 1: [1,1]
Tube 2: [0,0]
Tube 3: [2,2]
Tube 4: [0,0]

C:\Users\dodal\Downloads\Colored_Water_Sort\x64\Debug\Colored_Water_Sort.exe (process 13100) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 7.1. Actual console output for Test Case 1 — Official Example A.

Test Case 2 — Official Example B

Expected: No solution exists.



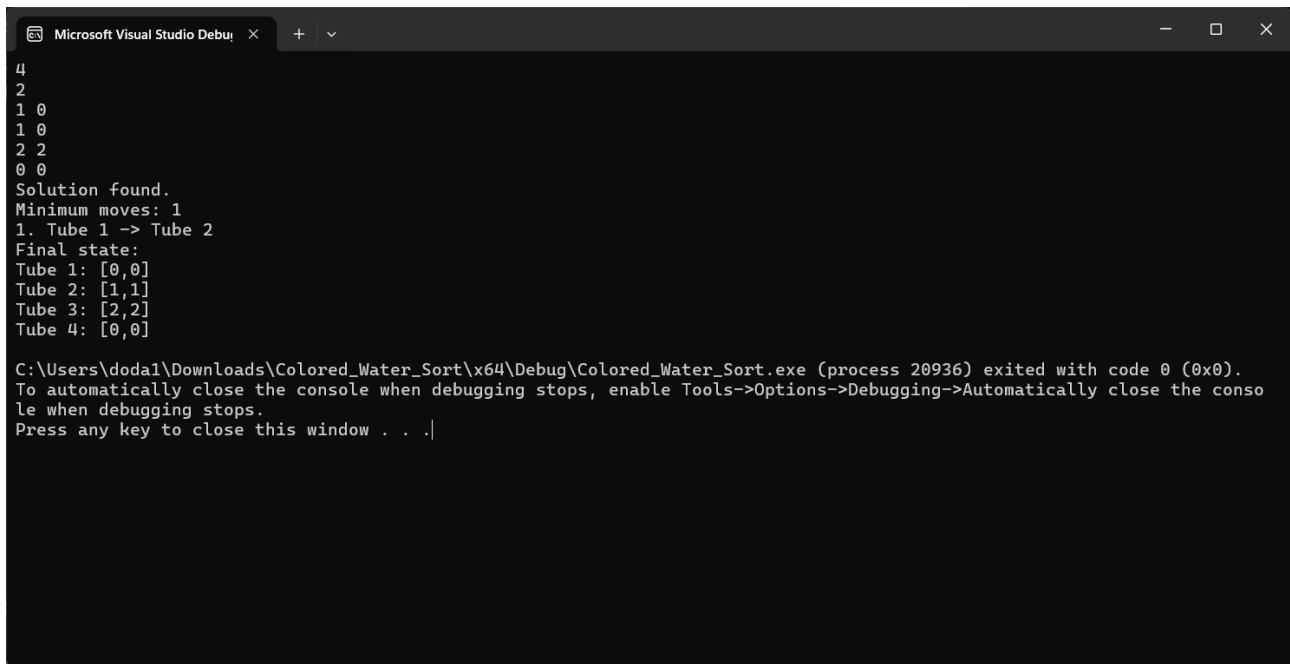
```
Microsoft Visual Studio Debug Console
2
2
1 2
2 1
No solution exists.

C:\Users\dodal\Downloads\Colored_Water_Sort\x64\Debug\Colored_Water_Sort.exe (process 21220) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 7.2. Actual console output for Test Case 2 — Official Example B.

Test Case 3 — Additional Test

Expected: Solution found with minimum 1 move.



```
4
2
1 0
1 0
2 2
0 0
Solution found.
Minimum moves: 1
1. Tube 1 -> Tube 2
Final state:
Tube 1: [0,0]
Tube 2: [1,1]
Tube 3: [2,2]
Tube 4: [0,0]

C:\Users\dodal\Downloads\Colored_Water_Sort\x64\Debug\Colored_Water_Sort.exe (process 20936) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figure 7.3. Actual console output for Test Case 3 — Additional Test.

8. GitHub Repository

The complete public GitHub repository containing the C++ source code, sample inputs, README, and project documentation is available at:

<https://github.com/mahmoudabdelazizmohammed/Colored-Water-Sort>

The repository is public so the source code and project materials can be reviewed.

9. Conclusion

The final implementation provides a console-based Colored Water Sort solver using vector-based state representation, BFS, visited-state checking, and parent tracking. It solves the official solvable example in three moves, reports no solution for the official unsolvable example, and passes an additional test case.